

L4

模型微调与 私有化大模型

4. 大模型微调详解

大模型微调编程

4.1.1 Hugging Face 生态概览

transformers

```
pip install transformers
```

模型架构、Tokenizer、Trainer，核心基础库

datasets

```
pip install datasets
```

标准化数据集加载与处理，支持流式读取大规模数据

peft

```
pip install peft
```

参数高效微调：LoRA、Prefix-Tuning、Adapter 等方法

trl

```
pip install trl
```

SFT Trainer、DPO Trainer，完整指令微调流水线

accelerate

```
pip install accelerate
```

多卡分布式训练、混合精度（BF16/FP16）透明抽象层

4.1.2 完整微调流程（七步走）

01

环境安装

`pip install transformers datasets peft trl accelerate`

02

加载模型 & Tokenizer

BF16半精度加载, `device_map=auto` 自动分配GPU

03

数据集构建

JSON→Dataset, `apply_chat_template` 格式化,
`train_test_split`

04

配置 LoRA

LoraConfig: `r=16`, `alpha=32`, `target_modules` 7层,
`get_peft_model`

05

设置训练参数

TrainingArguments:
`lr/epochs/batch/warmup/scheduler/bf16`

06

启动 SFT 训练

`SFTTrainer.train()`, 自动保存 checkpoint, TensorBoard日志

07

合并 LoRA 权重

`PeftModel + merge_and_unload()` → 保存完整模型供推理部署

● bash

核心训练库

```
pip install transformers datasets peft trl accelerate
```

PyTorch (根据 CUDA 版本选择)

```
pip install torch torchvision torchaudio
```

transformers

HuggingFace 模型加载与训练框架

datasets

高效数据集加载与处理工具

peft

Parameter-Efficient Fine-Tuning, LoRA 实现

trl

SFTTrainer / RLHF 训练封装

accelerate

分布式训练与混合精度加速

torch

深度学习计算引擎 (需匹配 CUDA 版本)

• python

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

model_name = "Qwen/Qwen1.5-1.8B-Chat"

tokenizer = AutoTokenizer.from_pretrained(model_name, trust_remote_code=True)

model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.bfloat16,    # BF16 半精度, 显存减半, 训练稳定
    device_map="auto",            # 自动分配 GPU/CPU
    trust_remote_code=True
)
```

- `torch_dtype=torch.bfloat16` — BF16 半精度, 推理和训练更稳定, 比 FP16 精度更高
- `device_map='auto'` — 自动分配显存, 多 GPU 环境下自动分片

• python

```
def format_chat_template(sample):  
    """将指令格式转换为对话格式 (Chat Template) """  
    messages = [  
        {"role": "system", "content": "你是一个专业的电商客服助手..."},  
        {"role": "user", "content": sample["instruction"]},  
        {"role": "assistant", "content": sample["output"]}  
    ]  
    text = tokenizer.apply_chat_template(  
        messages, tokenize=False, add_generation_prompt=False  
    )  
    return {"text": text}  
  
dataset = Dataset.from_list(raw_data)  
dataset = dataset.map(format_chat_template)  
dataset = dataset.train_test_split(test_size=0.1, seed=42)
```

- `apply_chat_template` — 将多轮对话转换为模型可理解的特殊 token 序列
- `train_test_split(test_size=0.1)` — 自动划分 90% 训练 / 10% 验证集

- python

```
from peft import LoraConfig, get_peft_model, TaskType
```

```
lora_config = LoraConfig(  
    task_type=TaskType.CAUSAL_LM,  
    r=16,  
    lora_alpha=  
32,  
    target_modules=[  
        "q_proj", "k_proj", "v_proj", "o_proj",  
        "gate_proj", "up_proj", "down_proj"  
    ],  
    lora_dropout=0.05,  
    bias=  
"none"  
)
```

```
model = get_peft_model(model, lora_config)  
model.print_trainable_parameters()
```

r = 16

LoRA 秩
控制可训练参数量

lora_alpha = 32

缩放因子
推荐设为 $2 \times r$

target_modules

注入目标层
Attention + FFN 全覆盖

lora_dropout = 0.05

Dropout 率
防止过拟合

► **OUTPUT** trainable params: 6,815,744 || all params: 1,841,213,440 || trainable%: **0.37%**

- python

```
training_args = TrainingArguments(  
    output_dir="./qwen-ecommerce-lora",  
    num_train_epochs=3,  
    per_device_train_batch_size=4,  
    gradient_accumulation_steps=4,  
    learning_rate=2e-4,  
    lr_scheduler_type="cosine",  
    warmup_ratio=0.03,  
    weight_decay=0.001,  
    bf16=True,  
    eval_strategy="steps", eval_steps=100,  
    save_steps=200, save_total_limit=3,  
    load_best_model_at_end=True,  
    report_to="tensorboard"  
)
```

`gradient_accumulation_steps=4`

等效 batch_size=16
节省显存同时保持效果

`lr_scheduler_type='cosine'`

余弦学习率衰减
训练后期更平滑收敛

`bf16=True`

BF16 混合精度训练
显存减半，训练更快

`eval_steps=100`

每 100 步评估一次
及时发现过拟合

`load_best_model_at_end`

自动加载最优 checkpoint
避免手动挑选模型

• python

```
from trl import SFTTrainer

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    dataset_text_field="text",
    max_seq_length=2048,
    packing=False
)

trainer.train()

trainer.save_model("./qwen-ecommerce-lora-final")
```

- SFTTrainer — trl 提供的监督微调封装，内部支持 PEFT / LoRA 无缝集成
- max_seq_length=2048 — 序列截断长度；packing=False 保证样本独立训练

• python

```
from peft import PeftModel

# 1. 加载原始基座模型
base_model = AutoModelForCausalLM.from_pretrained(
    model_name, torch_dtype=torch.bfloat16, device_map="auto"
)

# 2. 加载并附加 LoRA 适配器
merged_model = PeftModel.from_pretrained(
    base_model,
    "./qwen-ecommerce-lora-final"
)

# 3. 融合权重并释放 LoRA 模块
merged_model = merged_model.merge_and_unload()

# 4. 保存完整模型
merged_model.save_pretrained("./qwen-ecommerce-merged")
```

- `merge_and_unload()` — 将 LoRA 权重与基座融合, 消除推理额外开销
- 融合后的模型与普通模型无区别, 可直接用 vLLM / llama.cpp 等推理框架部署

完整流程回顾

1

环境安装

transformers · peft · trl · accelerate

2

加载模型

BF16 半精度 + 自动显存分配

3

数据格式化

Chat Template + 自动划分训练/验证集

4

配置 LoRA

r=16 · alpha=32 · 可训练参数 0.37%

5

训练参数

cosine 调度 · BF16 混合精度 · 梯度累积

6

SFT 训练

SFTTrainer · 自动保存最优模型

7

合并权重

merge_and_unload · 零推理开销部署

4.2.1 LoRA 核心参数详解

参数	含义	推荐值	影响说明
r (rank)	LoRA矩阵的秩，决定参数量	8 ~ 64	r越大参数越多，表达能力越强，显存消耗随之增加
lora_alpha	缩放系数，影响等效学习步长	2×r	实际缩放比例 = lora_alpha / r；通常设为2r保持稳定
lora_dropout	LoRA层Dropout比率	0.05 ~ 0.1	防止过拟合；数据量充足时可降至0.0
target_modules	注入LoRA的目标层名称	q/k/v/o/gate/up/down	覆盖层越多效果越好，参数量相应增加
bias	是否同时微调bias参数	"none"	通常设none；设lora_only仅微调LoRA层bias

4.2.3 LoRA r 值对比 & 数据相关参数

不同 r 值效果对比			
r 值	可训练参数	显存增量	适用场景
4	~230K	极小	快速验证、极少数数据
8	~460K	小	小数据集 (<1万条)
16	~920K	中	常规微调 (推荐起点)
32	~1.84M	较大	复杂任务、数据充足
64	~3.68M	大	高难度任务、接近全参

数据相关参数	
packing	将多个短样本拼接成一个序列，显著提升短文本数据集的GPU利用率
dataset_text_field	指定数据集中文本字段名称（如 'text'），SFTTrainer 依此读取
max_seq_length	超长序列将被截断；需根据任务最长输入+输出合理设置，影响显存
neftune_noise_alpha	NEFTune 噪声注入（推荐值5-15），可提升指令微调泛化能力

模型训练超参数选择

这些超参数决定了 一次训练中输入给模型的数据形态：

- **max_length**
 - 单条输入序列的最大长度（token 数）。太长会截断，太短会 padding。
 - 影响显存和训练速度。
 - 小模型 / 短文本任务（分类、QA）：128 ~ 512;
 - 中长文本（指令微调、对话）：512 ~ 2048
- **per_device_train_batch_size**
 - 每张 GPU 上一次送入模型的样本数量。
 - 显存小 → batch size 小（1~2），显存大 → 可以更大。
 - 4090 / A100（24GB~80GB）：2 ~ 16
 - 普通消费级 GPU（8GB~12GB）：1 ~ 2
- **gradient_accumulation_steps**
 - 梯度累积次数。
 - 比如 batch=2, steps=8 → 等效 batch=16。
 - 用来解决显存不足的问题。
- **eval_batch_size**
 - 验证时的 batch 大小，一般和训练一致或更大。

- 这些参数决定了 **模型如何学习**：
- **learning_rate**
 - 学习率，控制参数更新的步伐。太小 → 学得慢；太大 → loss 震荡或发散。
 - LoRA 通常用 $1e-4 \sim 2e-4$ 。
- **num_train_epochs**
 - 训练轮数。一个 epoch = 数据完整过一遍。
 - 数据多时通常只需要 1~3 轮。
- **warmup_steps / warmup_ratio**
 - 前几个 step 使用较小的学习率，再逐渐增大。防止训练初期不稳定。
- **weight_decay**
 - 权重衰减，避免参数过大导致过拟合。常用值：0.01。
- **optimizer**（通常用 AdamW）
 - 带有一阶/二阶矩估计，比 SGD 稳定。

这些参数决定了 训练过程的稳健性：

- **dropout / lora_dropout**
 - 在训练中随机丢弃部分输入，避免过拟合。
 - LoRA 里一般设 0.05 ~ 0.1。
- **fp16 / bf16**
 - 混合精度训练，减少显存占用。
 - bf16 更稳定，适合 A100/H100； fp16 适合 V100。
- **gradient_clipping(max_grad_norm)**
 - 限制梯度大小，避免梯度爆炸。
 - 常用值：1.0。
- **save_strategy / save_steps**
 - 定期保存模型（adapter 权重）。
- **evaluation_strategy / eval_steps**
 - 定期在验证集上计算 loss，防止过拟合。

模型训练超参数的优化

- 损失函数：

- 损失函数：Loss = 模型预测结果 与 真实答案 的差距。
- 训练的目标就是：不断让 loss 变小，说明模型预测得越来越准。
- 在 语言模型里，用的是交叉熵损失（Cross-Entropy Loss），它会对每个预测的 token 和真实 token 做比较。

- loss 在训练中的作用

- 训练时，每个 batch 都会计算一个 loss。
- 优化器会根据 loss 的大小和方向，调整模型的参数。
- 对比训练集和验证集的Loss的差别

•学习率 (learning_rate)

- 太高 → loss 震荡甚至发散。
- 太低 → 收敛太慢。
- 调法：从 $2e-5 \sim 2e-4$ 试，观察 loss 曲线。

•batch_size + gradient_accumulation_steps

- batch 太小 → 梯度波动大，loss 曲线抖动。
- batch 大 → 收敛更稳，但需要更多显存。

•dropout / lora_dropout

- 验证集表现比训练差很多 → 增大 dropout，缓解过拟合。

•训练轮数 (num_train_epochs)

- loss 还在明显下降就停了 → 可以再加 epoch。
- loss 已经收敛还继续训练 → 浪费资源，甚至过拟合。

TensorBoard 监测指标

启动命令: `tensorboard --logdir ./logs --port 6006` → 浏览器访问 `http://localhost:6006`

train/loss

✓ 健康

训练集损失，应持续下降并趋于平稳；急剧下降后平台期属正常

eval/loss

⚠ 关键

验证集损失，应与 train/loss 同步下降；若反弹上升则为过拟合信号

train/learning_rate

✓ 参考

学习率曲线，应先线性升温（warmup）再余弦下降至接近0

train/grad_norm

⚠ 关键

梯度范数，正常<5；持续>10 说明梯度爆炸，需减小lr或开启梯度裁剪

train/epoch

✓ 参考

当前训练轮次，用于横轴对齐；结合steps理解训练进度

常见训练问题诊断与处理

① 损失不下降 (Loss Stuck)

原因： 学习率过小 / 数据格式错误 / target_modules设置有误

处理： 学习率乘以10倍 → 检查chat_template格式 → 确认LoRA注入层名称

② 损失发散 (Loss Exploding)

原因： 学习率过大 / 未设置warmup / bf16与模型不兼容

处理： 学习率除以10 → 设warmup_ratio=0.05 → 开启max_grad_norm=1.0 梯度裁剪

③ 过拟合 (Overfitting)

原因： 训练轮数过多 / 数据量不足 / dropout过小

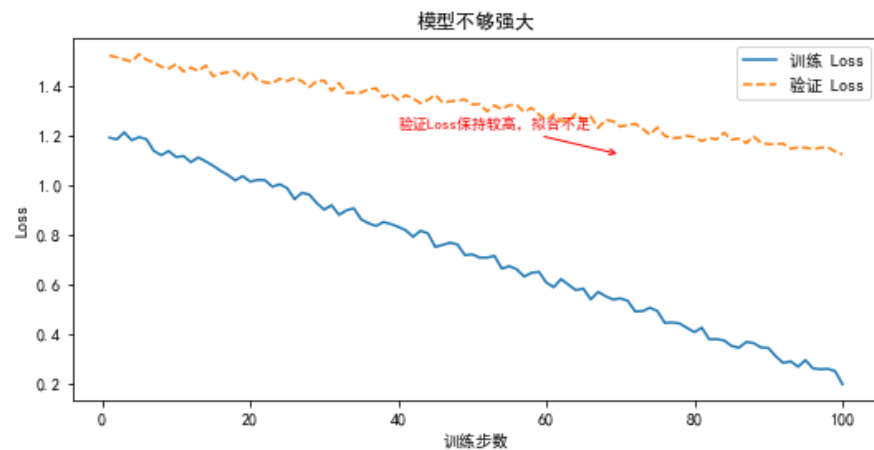
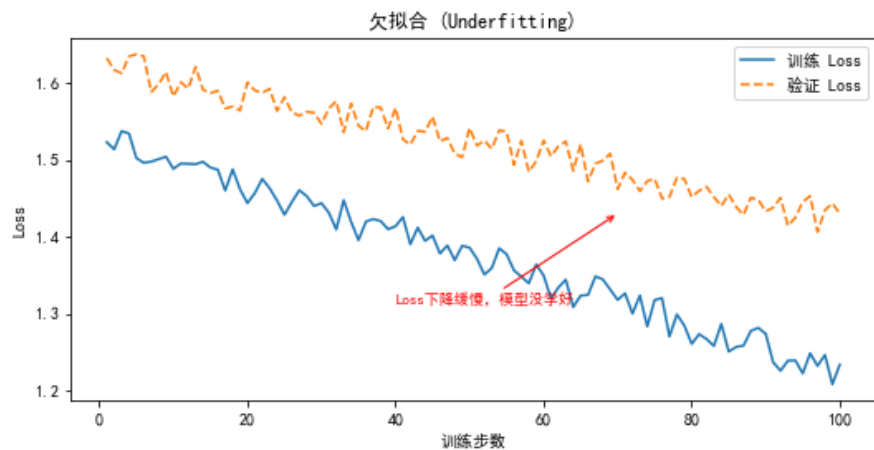
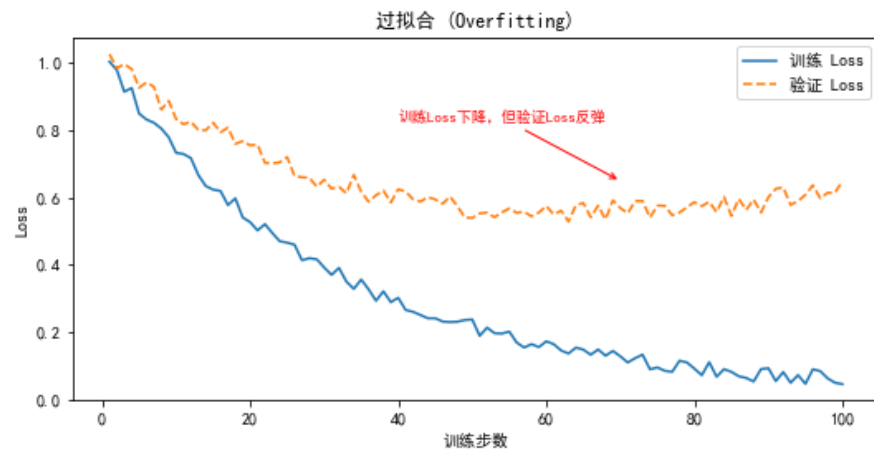
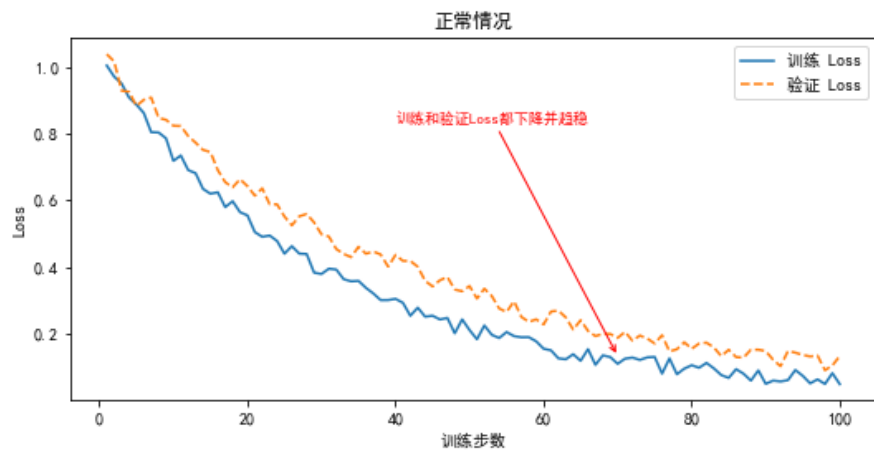
处理： 增大 lora_dropout → 增加 weight_decay → 减少 epochs → 扩充数据多样性

④ 显存溢出 (OOM)

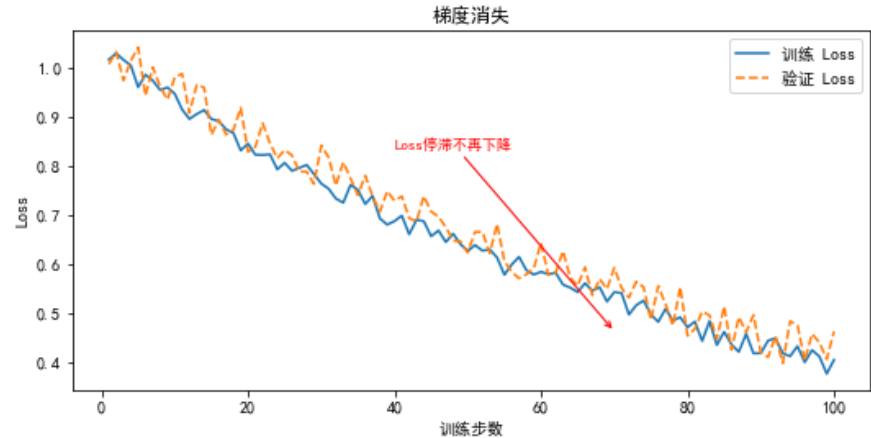
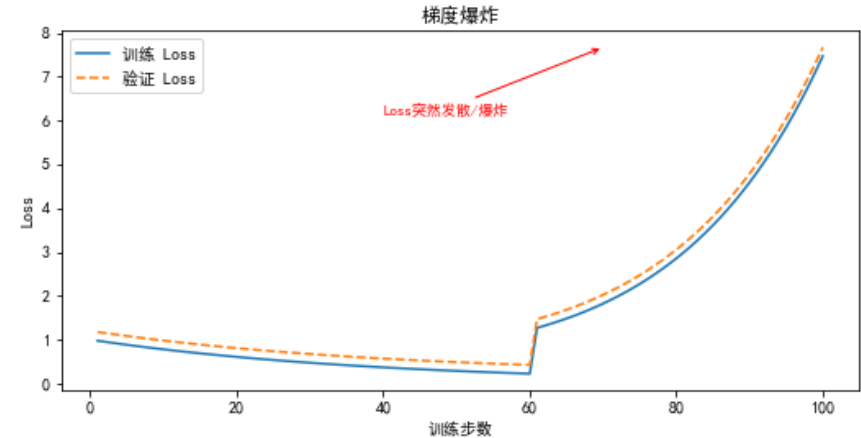
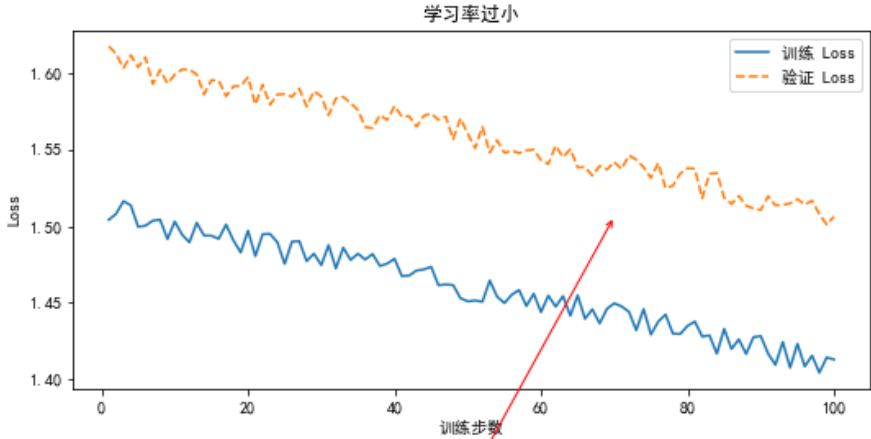
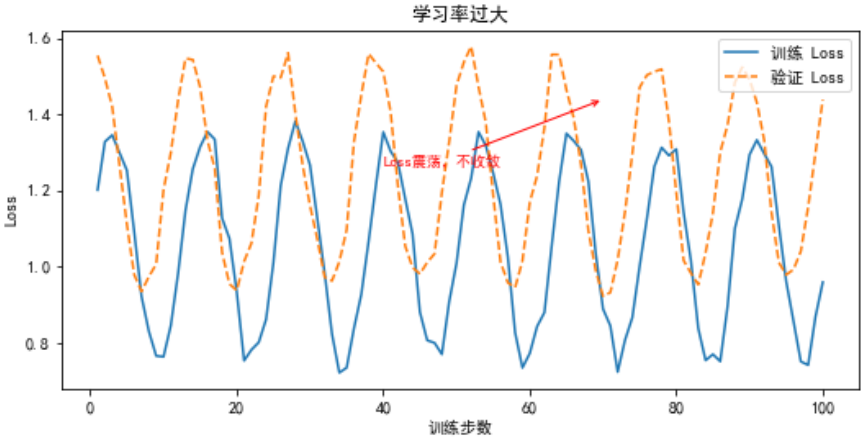
原因： batch_size过大 / 序列过长 / 模型本身过大

处理： 减小 batch_size → 增大 gradient_accumulation_steps → 开启 gradient_checkpointing → 减小 r 值

损失曲线示例



损失曲线示例



情况	现象	解决办法
正常	Loss 稳定下降	无需调整
过拟合	Train ↓ , Eval ↑	正则化、早停、数据增强
欠拟合	Loss 下降少	增强模型、加 epoch、调大学习率
模型不够强大	Eval Loss 高	换更大模型、改进数据
学习率过大	Loss 震荡	降低学习率、加调度器、梯度裁剪
学习率过小	Loss 下降慢	提高学习率、加调度器、加 epoch
梯度爆炸	Loss 突然发散	梯度裁剪、降低学习率、稳定优化器
梯度消失	Loss 停滞	更好激活函数、残差连接、更强模型

大模型评估方法

大语言模型评估维度与指标对照表

评估维度	典型任务/场景	常见基准/数据集	自动化指标	人工/其他方法
语言与任务能力	分类、推理、问答、翻译、摘要	GLUE / SuperGLUE、MMLU、BIG-Bench、GSM8K、HumanEval、MBPP、Flores-200	Accuracy、F1、Exact Match、BLEU、ROUGE、BERTScore	专家打分，A/B 测试
专业领域能力	医学、法律、金融等	MedQA、PubMedQA、LawBench	Accuracy、F1	专家人工评估
交互与对话质量	多轮对话、客服问答	Chatbot Arena、DialEval	无固定自动化指标（常用 BERTScore / Embedding 相似度）	人工评价（流畅度、相关性、一致性、礼貌性）
安全性与可信度	防止有害输出、虚假信息	红队测试集（harmful prompts）、TruthfulQA	幻觉率、毒性检测 (Toxicity Score)	人工红队、事实核查
鲁棒性与泛化	拼写错误、跨领域迁移、对抗攻击	AdvGLUE、NoiseQA	Robust Accuracy	对抗提示实验

一、词面重合类 (Lexical Overlap)

看表面匹配度 — 计算字词层面的重合度

💡 **核心思想：** 把模型输出和参考答案当成两串文字，看它们在字词层面重合多少。计算快、实现简单、复现性强。

BLEU

计算 n-gram (连续n个词) 的精确匹配率
2-gram 即「连续两个词」有多少对得上

◆ 经典场景：机器翻译

ROUGE

更关注召回率：参考答案的内容你覆盖了多少
还会看最长公共子序列 (LCS)

◆ 经典场景：文本摘要

METEOR

在 BLEU 基础上改进：

- 词干还原 (running $\approx$ run)
- 同义词表 • 词序考量

◆ 更宽容、更接近人类判断

⚠ **局限性：** 只认字面，不认语义。「这部电影很棒」和「这片子真不错」在 BLEU 眼里可能毫无关系。
开放式生成任务上参考价值有限，需结合语义指标使用。

二、语义相似度类 (Semantic Similarity)

看意思接近度 — 用向量空间度量语义距离

💡 **核心思想：**用预训练语言模型把句子编码成向量，在语义空间里比较两者的距离，能识别同义改写和表达变体。

BERTScore

用 BERT 分别编码候选答案和参考答案
然后做 token 级别的向量相似度匹配

◆ 广泛适用，工具链成熟

MoverScore

借鉴 Word Mover's Distance (词移距离)
把文本词向量分布「搬运」成另一段的最小代价
与最优传输 (Optimal Transport) 有关

◆ 数学优雅，对语序更敏感

BLEURT

基于 BERT 做有监督微调
训练数据是大量人类标注的质量评分

◆ 和人类判断相关性更高

⚠ **局限性：**需要加载大模型，算力成本较高。不同模型版本会影响分数的可比性。
在对话生成、开放问答等任务上优势明显，是词面类指标的有力补充。

三、分类 / 问答类 (Classification & QA)

看答案对错 — 有标准答案就直接比对

💡 **核心思想：** 任务有明确的「标准答案」，那就直接比对是否正确。客观、严格、易解释，适用于有明确对错的任务。

Accuracy (准确率)

预测正确的样本占总样本的比例

- 样本类别均衡时效果好
- MMLU 知识测评的标准指标
- ◆ 最直观的正确率衡量

F1 Score

Precision (预测为正中真正的比例)
与 Recall (真正中找出多少) 的调和平均

- 类别不平衡时必用
- 医学诊断、垃圾邮件识别等场景
- ◆ 兼顾漏判和误判

Exact Match (EM)

预测结果与参考答案完全一致才算对
一字之差都不行

- SQuAD 等抽取式问答的常见指标
- 信息抽取、NER 任务
- ◆ 最严格的精确匹配

⚠ **局限性：** 对开放式生成束手无策 —— 让它评价一首诗，它只会说「和参考答案不一样，零分」。

适用范围：MMLU 知识测评 · 抽取式问答 · 信息抽取 · 文本分类。

四、事实性与一致性类 (Factuality & Consistency)

看有没有「幻觉」 — 检测输出是否忠实于原文或事实

💡 **核心思想：** LLM 突出问题是「一本正经地胡说八道」 (hallucination)，这类指标专门检测输出是否忠实于原文或事实。

FactCC

基于 NLI (自然语言推理) 训练的分类器
判断摘要中的陈述是否能被原文蕴含

示例：

原文说「会议周一召开」

摘要写成「会议周五召开」

→ FactCC 能将其识别为事实错误

QAGS

- ① 针对模型生成的摘要自动出题
- ② 让模型分别基于「原文」和「摘要」回答
- ③ 对比两组答案是否一致
- ④ 不一致之处 = 潜在事实错误点

⚠ **现状：** 方法仍不够成熟，依赖底层 NLI/QA 模型的质量，更多出现在研究论文中。
工业界落地仍在探索，但对摘要、RAG、知识问答任务很有价值。

该怎么选？实用建议

四类指标互补而非互斥，成熟方案 = 多指标组合 + 人工抽检



翻译任务

BLEU

BERTScore

→ 字面精确 + 语义双重把关



摘要任务

ROUGE

BERTScore

FactCC

→ 覆盖度 + 语义 + 事实性三维评估



知识问答

EM

F1

→ 严格匹配 + 宽松匹配各看一个



开放对话

BERTScore

BLEURT

人工评估

→ 自动指标有天花板，人工是金标准

自动指标是人类判断的近似 —— 「金标准」仍是人。这也是 RLHF 和 LLM-as-a-Judge 兴起的根本原因。

全局总览：四大类指标速查表

一张表掌握核心要点

类别	代表指标	核心维度	优势	适用任务
词面重合 (Lexical)	BLEU / ROUGE METEOR	n-gram 精确/召回率	快速、可复现	翻译 · 摘要
语义相似 (Semantic)	BERTScore MoverScore / BLEURT	向量语义距离	识别同义改写	对话 · 问答
分类/问答 (QA Metrics)	Accuracy F1 / EM	正确/错误匹配	客观严格	知识测评 · NER
事实一致 (Factuality)	FactCC / QAGS	忠实度 / 幻觉检测	识别事实错误	摘要 · RAG

AI辅助编程

什么是 CodeBuddy & 核心功能

腾讯云出品的 AI 编程助手，集成于 VS Code 等主流 IDE，专为中文开发者优化。课程引入目的：降低编程门槛，让学员聚焦模型原理而非语法细节。

代码补全

行级/块级智能补全，感知上下文意图，对PyTorch/Transformers框架有专项优化

对话式编程

用中文描述需求直接生成代码；支持代码解释、重构、bug修复、注释生成

文档生成

选中代码一键生成中文注释和docstring，显著降低文档维护成本

单元测试

自动生成pytest测试用例，覆盖正常输入、边界条件、异常情况

终端命令助手

自然语言转shell命令；如：查看GPU使用率 → `nvidia-smi -l 1`

典型使用场景 & AI编程工具对比

使用场景	传统方式	使用 CodeBuddy
编写数据预处理脚本	查文档 + 手写，约30分钟	描述需求，2分钟内生成
理解 LoRA 配置参数	搜索英文文档，费时费力	选中代码，一键中文解释
调试 OOM 报错	搜索 Stack Overflow	粘贴报错，直接给出解决方案
生成评估代码	参考论文代码实现	描述评估指标，自动生成完整脚本
编写训练回调函数	手写模板代码	描述功能需求，自动生成回调类

工具	厂商	中文支持	国内访问	最适场景
CodeBuddy (推荐)	腾讯云	★★★★★	✅ 流畅	国内中文AI开发
通义灵码	阿里云	★★★★★	✅ 流畅	阿里云生态
文心快码	百度	★★★★★	✅ 流畅	百度生态
GitHub Copilot	Microsoft	★★★	⚠️ 需代理	英文开源项目
Cursor	Cursor AI	★★★★	⚠️ 需代理	全栈开发

实践：大模型微调实践

1. Swift 可视化界面练习 （P03 实践 4）
2. 微调程序及训练过程监测 （P03实践 5）
3. 模型合并与模型评估
4. 大模型辅助编程
5. 综合项目程序介绍